

Wave 5.0 — Generation Latency & Reliability Audit

Scope: AIHomeArchitect backend /generate pipeline.

Mode: Audit only — no code changes, no optimizations applied.

Freeze status: Intelligence-foundation freeze fully respected.

0. Methodology & honesty disclaimer

This is a static audit grounded in:

1. backend/docs/PERFORMANCE_ANALYSIS.md (Wave 4.4.2 — already instrumented + measured)
2. backend/docs/FULL_SYSTEM_AUDIT.md (Wave 4.5.2 — system-by-system classification)
3. backend/docs/PROMPT_FLOW_ANALYSIS.md (Wave 4.5.2 — V1 prompt budget arithmetic)
4. backend/KNOWN_LIMITATIONS.md (L1–L4, by-design tradeoffs)
5. Static analysis of main.py /generate, retry_classifier.py, generation_profiles.py, performance_obs_server.py

The production log in this tree is empty (0 bytes), so live wallclock data cannot be mined here. Where numbers come from the 4.4.2 instrumentation they are cited; where they are inferred from code structure that is labelled. Anything that would need a live A/B to confirm is flagged "needs live measurement".

1. Current generation pipeline map (post-4.7.0, current production)

```

POST /generate
├─ Step 1 resolve_source / image_fetch    -- [PERF] image_fetch
│   * V1: original_image_url (uploaded photo)
│   * V2+ default: LATEST vision URL (Wave 4.7.3)
│   * V2+ optional: ORIGINAL or SPECIFIC_VERSION
│
├─ Step 2 vision_analysis (gpt-4o-mini, detail=high)
│   * PROD: runs at V1 AND V2+
│   * mobile_mvp_baseline: SKIPPED at V1 only (Wave 4.7.0)
│   * non-fatal: empty room_description if it fails
│   * instrumented [PERF] vision_analysis
│
├─ Step 3 structural_identity
│   * if client token present -> from_token() (free)
│   * else if V1 -> extract_from_description (free, deterministic)
│   * else V2+ -> also extract_from_description (free)
│   * Wave 4.7.2 client persists token after V1 -> V2+ skips parse
│
├─ Step 4 Resolve let_ai_decide (classify_room) + surprise_me_flag
│   * Both depend on room_description; both V1-only by client contract
│
├─ Step 5 classify_intent + classify_transformation + classify_edit_mode
│   * Local CPU (<5 ms each), regex/dict
│   * V1 -> FIRST_VISION; V2+ -> STYLE_REFINEMENT (compact path)
│
├─ Step 6 compose_generation_prompt (composer.py, ~5-50 ms)
│   * FIRST_VISION: task -> contract -> source -> DNA -> completeness -> wow -> realism
│   * STYLE_REFINEMENT: compact path, refinement_memory layer
│
├─ Step 7 _detect_output_size  -> 1536x1024 | 1024x1536 | 1024x1024
│
├─ Step 8 build_structural_mask (PIL, ~30-80 ms, run_in_executor)
│   * Only FIRST_VISION + STYLE_REFINEMENT
│   * Skipped if profile.use_mask == False
│
├─ Step 9 openai.images.edit  <-- DOMINANT LATENCY (30-90 s/attempt)
│   * quality=high, input_fidelity=high, max_retries=0 (SDK)
│   * App-level retry loop, max 3 in PROD, classified per failure
│   * instrumented per-attempt [PERF] openai_api
│
├─ Step 10 supabase_upload (~500 ms - 3 s)
│   * instrumented [PERF] supabase_upload + [PERF SUMMARY]

```

Full instrumentation already exists in `performance_observer.py::PipelineTimer` and emits a single `[PERF SUMMARY]` line per request with every stage, cost and cost_risk tier.

2. Latency breakdown (from existing 4.4.2 instrumentation)

Stage	Typical	Worst	Share of	Notes
image_fetch	500-2000 ms	5000 ms	~3%	Supabase CDN
vision_analysis (gpt-4o-mini)	3000-8000 ms	15000 ms	~10%	non-fatal
structural_identity	<5 ms	<50 ms	0%	client token -> 0; else regex
classifiers (x3)	<5 ms each	<50 ms	0%	local CPU
prompt_composition	5-50 ms	200 ms	0%	local CPU
mask_generation	30-80 ms	200 ms	0%	PIL, async
openai.images.edit (x1)	30000-90000 ms	120000 ms	~85%	dominant
supabase_upload	500-2000 ms	5000 ms	~2%	

TOTAL (1 attempt)	35-100 s	140 s	
TOTAL (2 attempts)	90-180 s	300 s	retry scenario

Conclusion. ~85% of typical wallclock is the OpenAI call itself. Everything else combined is at most 15%. The single highest-impact reduction would be eliminating one OpenAI attempt (retry -> first-shot success), not shaving the local pipeline.

3. Reliability breakdown (current state)

- `max_retries=0` on the OpenAI SDK — prevents 3 x SDK x 3 app = 9 hidden retries. Already correct.
- `retry_classifier.classify_for_retry` — comprehensive taxonomy (see §6).
- App retry loop — PROD `max_attempts=3`, DEV / MVP `max_attempts=1`.
- **No idempotency** — `client_request_id` is **logged but NOT used for deduplication** (cited in PERFORMANCE_ANALYSIS.md §Remaining Risks #4). A duplicate retry from the client re-runs the entire pipeline including a fresh OpenAI call.
 - **L4 reconciliation 90 s ceiling** — by-design tradeoff (KNOWN_LIMITATIONS).
 - **No explicit timeout on `images.edit`** — SDK default ~600 s. First-attempt failures at ~60 s are likely server-side (cited in 4.4.2 §Remaining Risks #1).

4. V1 vs V2/V3 recomputation audit (KEY FINDING)

Stage	V1 cost	V2+ cost	Recomputed at V2+ but unused?
<code>image_fetch</code>	0.5-2 s (original)	0.5-2 s (latest vision)	No — different image
<code>vision_analysis</code>	3-8 s	3-8 s (still runs)	YES (largely)
<code>structural_identity</code> (token-based)	parse, ~ms	0 ms (token reused)	Already optimised (Wave 4.7.2)
<code>classify_room</code> / <code>surprise_me</code>	runs if flagged	not used (V1-only contract)	n/a (flags not sent at V2+)
<code>classify_edit_mode</code>	<5 ms	<5 ms	No (mode changes V1 -> V2+)
<code>prompt_composition</code>	~30 ms (full FV)	~10-20 ms (compact SR)	No — different prompt
<code>mask_generation</code>	~50 ms	~50 ms	No — different source image
<code>openai.images.edit</code>	30-90 s	30-90 s	No — actual generation
<code>supabase_upload</code>	0.5-3 s	0.5-3 s	No — different result

Vision analysis at V2+ is the standout finding. Its outputs are:

1. **SOURCE_SPACE prompt section** — **removed at Wave 4.6.1** (per `generation_profiles.py` comment: "Wave 4.6.1 already removed SOURCE_SPACE from the FV prompt, so room_description is unused for FV anyway").
2. **classify_room (let_ai_decide)** — V1-only by frontend contract (Wave 4.8.5 sends `letAiDecide` only when `isFirstVision`).
3. **surprise_me** — same V1-only contract.
4. **structural_identity parse** — only when client token is absent (post-Wave 4.7.2 the token is always present at V2+).
5. **anchor_detector (STYLE_REFINEMENT)** — uses `room_description` to detect bay windows, multi-zone, columns. **This is its one remaining V2+ consumer.**

At V2+, vision analysis consumes 3-8 s of wallclock primarily to feed anchor_detector. Per Wave 4.5.2 audit, anchor outputs are 0-185 chars of additional contract text.

5. Cacheability analysis

Artefact	Stability	Recompute cost	Cache fit	Verdict
`structural_identity_token`	Per-session, set at V1	parse <5 ms or 0	Already cached client-side (Wave 4.7.2)	DONE
`room_description` (vision_analysis)	Per-(session, source-image)	3-8 s	Per-session at V1; per-image at V2+	SAFE CACHE — see §9 #1
Source image bytes (CDN)	Per-URL, immutable	0.5-2 s	CDN cache headers + server-side LRU	SAFE if URL-keyed — small win
Structural mask (PIL output)	Per-source-image	30-80 ms	Per-URL LRU	SAFE but small payoff (~50 ms)
Composed prompt	Per-(atmosphere, room, ...)	5-50 ms	High-dim cache key, low payoff	DON'T BOTHER
atmosphere DNA blocks	Per-(atmosphere, room) constants	0 (already const)	n/a	Already free
OpenAI generation output	Per-(prompt, image, params)	30-90 s	Idempotency-keyed on `client_request_id`	CRITICAL — DANGEROUS-IF-WRONG — see §6

Dangerous caches (do not implement).

- Caching the generated image under any key that is not the full request hash (silent quality drift).
- Caching across sessions / users (privacy + freshness).
- Caching the prompt budget arbitration (it is free; cache is pure overhead).

6. Retry taxonomy (codified in `retry\_classifier.py` — already complete)

Exception	Verdict	Reason	Notes
`AuthenticationError`	NON_TRANSIENT	openai-auth-failure	Fail fast
`PermissionDeniedError`	NON_TRANSIENT	openai-permission-denied	Fail fast
`NotFoundError`	NON_TRANSIENT	openai-not-found	Fail fast
`UnprocessableEntityError`	NON_TRANSIENT	openai-unprocessable	Validation issue
`RateLimitError`	TRANSIENT	openai-rate-limit	Retry
`InternalServerError`	TRANSIENT	openai-server-error	Retry
`APIConnectionError`	TRANSIENT	openai-connection-error	Retry
`APITimeoutError`	TRANSIENT	openai-timeout	Retry
`httpx.RemoteProtocolError`, `httpx.ConnectError`	TRANSIENT	transport-disconnect	Retry — common at ~60 s
`httpx.Timeout*`	TRANSIENT	transport-timeout	Retry

`httpx.HTTPStatusError` 5xx / 4xx	TRANSIENT / NON_TRANSIENT	by status	Correct split
`ConnectionResetError`, `ConnectionAbortedError`, `OSError`	TRANSIENT	stdlib net	Retry
`BadRequestError` (content policy / bad input)	NON_TRANSIENT	handled upstream	Cited in classifier docstring
Unknown exception	UNKNOWN -> retry once	flagged for monitoring	Conservative

Idempotency risk. The current loop retries the same `client_request_id`, so a re-attempt is logically idempotent at the request layer — but the OpenAI image-edit call is not idempotent on the OpenAI side, so two near-simultaneous attempts (e.g., client retry after our retry already started) can produce **duplicate billed images**. `client_request_id` is logged but not used to dedup — that is a real risk left unaddressed.

7. Resolution strategy analysis

Current PROD: aspect-matched, quality=high.

- 1536x1024 (landscape): \$0.190
- 1024x1536 (portrait): \$0.190
- 1024x1024 (square): \$0.080

Per Wave 4.5.2 FULL_SYSTEM_AUDIT, both quality=high and aspect-matched sizing are classified **ESSENTIAL** (System 2 and System 3). Lowering quality risks visual regression that the audit specifically calls "exactly the premium quality we're selling." Forcing a square output flips room proportions.

Existing fallback already coded but currently uncertain. `mobile_mvp_baseline` profile (Wave 4.7.0) drops quality to medium and omits `input_fidelity`, with all other prompt architecture intact. The 4.4.2 doc DEFERRED any high -> medium switch pending a benchmark; that benchmark has not been completed here.

Honest recommendation (no live data to override the 4.5.2 ESSENTIAL classification).

- **Keep PROD as-is.** No blind resolution change.
- **Run BENCHMARK_PROTOCOL.md** against PROD vs `mobile_mvp_baseline` to get scored A/B data. Only then consider V2+-specific quality differentiation.
- **Do not** introduce a V1 / V2+ resolution split before this benchmark; chained V2+ at lower quality drifts compoundingly.

8. Risk map

Risk	Likelihood	Impact	Mitigation status
L1 — Architectural identity preservation	Medium	Quality	Deferred to Wave 4.3.x (KNOWN_LIMITATIONS)
L4 — 90 s reconciliation gives up	Low-Med	UX	By design; "took longer than expected" message
OpenAI 60 s server-side timeout -> retry -> 2x cost	Med	Cost + UX	Smart retry classifier in place; no explicit <code>`images.edit`</code> timeout
Duplicate billed images from client double-tap or reconnect during retry	Low-Med	Cost + correctness	<code>`client_request_id`</code> logged but unused — gap

Retry storm via SDK + app loop	Very low	Cost	`max_retries=0` SDK already in place
V2+ vision analysis recomputation	High frequency	3-8 s / refinement perceived latency	Not addressed (this audit's main finding)
Freeze breakage from blind optimisation	n/a	Architectural	Freeze contract holds; this audit changes nothing
`BadRequestError` from content policy	Low	UX	Already handled upstream (non-transient, fail-fast)
Memory blow-up on huge sources	Very low	Stability	PIL ops in `run_in_executor`; not a real bottleneck
`room_description` cost on retry	Low	Cost	Vision called once per request (NOT per OpenAI attempt)

9. Ranked optimisation opportunities

#	Opportunity	Impact (typical)	Complexity	Risk	Verdict
1	Skip `vision_analysis` for V2+ (iteration > 1) when no V2+ consumer needs it; gate the same way the V1-mobile_mvp_baseline already does	-3 to -8 s per V2+ (~10% wallclock per refinement)	Low (one `if`)	Low — anchor_detector is the only V2+ consumer; if we choose to keep that, gate to "skip only if structural_identity token present AND anchor-heavy detection not required"	STRONG QUICK WIN
2	Honour `client_request_id` as an idempotency key (in-memory short-TTL cache of {request_id -> in-flight Future / final payload}); duplicates piggyback	Prevents 2x cost + collapses double-taps	Low-Med (small dict, TTL)	Low if scoped to in-process; would not survive worker restart, but eliminates the common client-retry double-charge	STRONG QUICK WIN
3	Set explicit `timeout=` on `openai.images.edit` based on real PROD P95 once measured (e.g. 75 s)	Stops occasional hangs near 600 s	Low	Low if conservative	SAFE — needs PROD timing first
4	Frontend perceived-latency: in `_LoadingBubble` tighten cadence + add a non-fake stage cue when backend `[PERF] vision_analysis` completes (small SSE / poll endpoint)	UX feel only	Med (new endpoint)	Low	DEFER — out of pure-backend scope; UX wave material
5	Cache `room_description` per (session_id, source_image_url) so a V2+ re-source from the original photo reuses V1's description	-3 to -8 s on the rare V2+ ORIGINAL re-source	Med (Supabase row or process cache + invalidation)	Med (staleness if image rewritten)	DEFER — payoff narrow; subsumed by #1 if vision is skipped on
6	CDN / HTTP cache headers on Supabase image URLs (server-side LRU on URL bytes)	-0.5 to -2 s per fetch	Low	Low-Med (image bucket policy)	SAFE, SMALL
7	Mask cache (per source-image URL, PIL output bytes)	-30 to -80 ms	Low	Low	TOO SMALL TO PRIORITISE

8	Run benchmark for `mobile_mvp_baseline` before any quality/fidelity change	Information only; gates future cost cuts	Med (manual evaluation)	n/a	PRECONDITION for #11
9	Vision-analysis token budget cut (220 -> 120 max_tokens)	Maybe -1 to -3 s on	Low	Unknown (deferred in 4.4.2)	NEEDS BENCHMARK
10	Parallelise vision + mask (both depend only on image_bytes)	-30 to -80 ms (mask becomes free)	Low	Low	TOO SMALL TO PRIORITISE unless #1 not done
11	PROD `quality=high` -> `medium` for V2+ refinements	-2.7x per V2+ image cost + possibly	Med	HIGH (visual regression on chained refinements)	DO NOT DO without benchmark
12	Reduce output size to 1024x1024	Cost cut	Low	HIGH (proportion distortion — System 3 ESSENTIAL)	DO NOT
13	Lower `input_fidelity`	Cost / speed	Low	CRITICAL (geometry drift — System 1 ESSENTIAL)	DO NOT

10. Recommended Wave 5.x execution roadmap

This audit is Wave 5.0 (read-only). The following implementation order minimises risk while front-loading the biggest wins.

- **Wave 5.1 — Skip V2+ vision analysis (Opportunity #1).** Single guarded if; instrument before / after with the existing [PERF SUMMARY] to measure actual V2+ wallclock reduction. Decide first whether to keep anchor_detector at V2+ (if yes, gate the skip on structural_identity token presence AND edit_mode in {STYLE_REFINEMENT}; if no, unconditional V2+ skip). Modifications limited to the gating boolean and log lines; no frozen-module internals changed. **Expected: -3 to -8 s per refinement.**
- **Wave 5.2 — Idempotency on `client\_request\_id` (Opportunity #2).** In-process dedup; collapses duplicate user retries into one OpenAI call. Closes the double-billing gap. Pure addition, no semantic change.
- **Wave 5.3 — Explicit `images.edit` timeout (Opportunity #3).** Set to a conservative P95 once 5.1 + 5.2 PROD data exists. Pure stability win.
- **Wave 5.4 — Run BENCHMARK_PROTOCOL (Opportunity #8).** PROD vs mobile_mvp_baseline. Pure measurement.
- **Wave 5.5 — Conditional Opportunity #11 (V2+ quality differentiation) ONLY** if 5.4 scores show acceptable drift. Otherwise the roadmap stops at 5.3.
- **Wave 5.6 — UX-perceived latency (Opportunity #4).** Out of pure backend scope; a frontend wave (e.g. stage-cue via SSE / polling). NOT Wave 4.11 territory; safe to do in parallel.

11. Safe quick wins vs dangerous optimisations

Safe quick wins (low risk, sub-Wave each).

- Skip vision_analysis on V2+ (Opportunity #1).
- Honour client_request_id for dedup (Opportunity #2).
- Set explicit images.edit timeout once measured (Opportunity #3).

- HTTP / CDN cache headers on Supabase (Opportunity #6).

Dangerous "optimisations" — do not do without explicit benchmark gates.

- Lower quality globally (Opportunity #11 without #8 benchmark): catastrophic-visual risk; flagged ESSENTIAL by 4.5.2 audit.
- Reduce output size to 1024x1024 (#12): geometry distortion.
- Lower input_fidelity (#13): geometry drift; flagged ESSENTIAL.
- Cache generated images outside a full request hash key: silent quality drift / cross-user contamination.
- Async fire-and-forget Supabase upload: hides upload failures from users.
- Bump SDK max_retries from 0: 9x hidden retry storm risk.

Final summary

The backend is architecturally strong and already comprehensively instrumented. The dominant ~85% of wallclock is the OpenAI `images.edit` call itself — which cannot be shortened without compromising quality. **The single highest-impact, lowest-risk Wave-5 opportunity is skipping ``vision_analysis`` on V2+ refinements**, where its 3-8 s cost is largely paid for nothing post-Wave-4.6.1. Pair it with idempotency on `client_request_id` to close the duplicate-billing gap, then measure before any quality knob is touched. **No prompt-semantic changes, no freeze breakage, no resolution change recommended at this time.**

Audit complete. No code modified.